

Identifying the LLM behind a response

CNN and recurrent neural network attribution on TXD-22

Team leader and sole member: Sangaa Chatterjee

Abstract

For this project, I wanted to see whether a machine learning model could tell which LLM wrote a response just by reading the text. I used the TXD-22 dataset and focused on three labels: ChatGPT, Claude, and Gemini. After cleaning and balancing the data, I had 8,901 responses to 2,967 prompts. I kept all three responses to a given prompt in the same data split so the models would not see that prompt during both training and testing. I trained a CNN and a bidirectional LSTM from scratch to compare how well they could identify the source.

On the 1,338 test responses, my CNN reached 83.3% accuracy and my LSTM reached 70.3% when they used response text alone. When I gave them only the prompts, both reached 33.3%, which is chance level for three balanced classes. I also tested whether including the prompt helps, and I looked at whether punctuation and model names affected the predictions. My results show clear patterns in this dataset, but I cannot claim that the models would identify every future version of these LLMs.

Questions covered

Question	Coverage
RQ1: Identify the generating LLM	CNN and LSTM; held-out accuracy, macro-F1, class metrics and confusion matrices.
RQ2: Does the prompt help?	Prompt only, response only, combined input and matched response-length control.
RQ4: What distinguishes models?	Style summaries and two retrained ablations. Extra-credit investigation.
RQ3: Cross-domain transfer	Not claimed. The CSV does not provide authoritative task/domain labels.

I completed the main experiments for RQ1 and RQ2 and investigated RQ4 for extra credit. I did not attempt RQ3 because this dataset does not have reliable task-category labels. The code and saved results are included in the accompanying project files.

1. Problem and dataset curation

I set this up as a three-class classification problem: given a piece of text, my model predicts whether its dataset label is ChatGPT, Claude, or Gemini. I used the dataset's Source column as the label, Question as the prompt, and Answer as the response. I left out platform names such as Poe and Perplexity because a platform can host different underlying models, and I wanted to compare LLM families instead.

Provenance and scope

I downloaded TXD-22 from the Kaggle link provided in the course materials on October 8, 2026 (listed version 2). The file had 66,000 rows across 22 sources, including 3,000 rows for each of the three labels I selected. An important limitation is that I could not verify the exact model versions or generation settings from the CSV. For that reason, throughout this report I treat ChatGPT, Claude, and Gemini as dataset labels rather than verified model identities.

Curation stage	Count / handling
Original dataset	66,000 rows; 22 categories
Selected families	9,000 rows; 3,000 per family
Missing/empty text	Removed if present
Privacy pattern matches	1 row removed
Repeated normalized answers	18 rows removed (all occurrences)
Complete prompt triplets retained	2,967 prompts; 8,901 responses

For cleaning, I standardized prompts by converting them to lowercase and removing extra whitespace. I kept no more than one response per prompt and LLM family, then selected prompts that had responses from all three families. I split the data by prompt instead of splitting individual rows. This was important because otherwise the same question could appear in both training and testing. I also checked that normalized prompts and exact normalized answers did not overlap across splits. Similar but not identical prompts may still appear in different splits.

Split	Prompt groups	Responses per family	Total
Training	2,076	2,076	6,228
Validation	445	445	1,335
Test	446	446	1,338

The downloaded file did not decode as UTF-8, so I used Latin-1 to read it without dropping rows. That can leave some encoding problems in the text. I screened for email-address and US Social Security number patterns, although this is not a complete privacy check. I have not included the raw dataset text in the report. The Kaggle reuse license was listed as unknown, so I would confirm permissions before redistributing that text. I saved file and split hashes to make the data preparation easier to check.

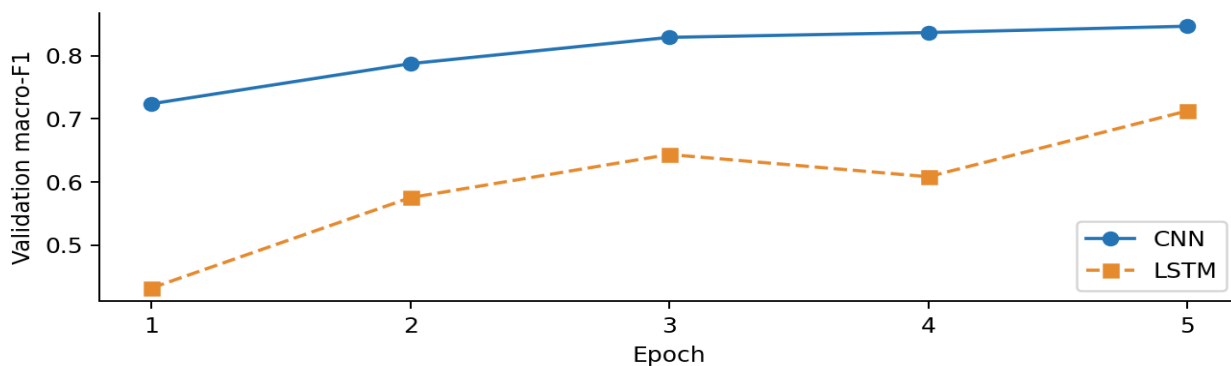
2. Models and training procedure

I implemented both required models in PyTorch. Before training, I converted text to lowercase and split it into words and punctuation tokens. I built each vocabulary from training data only and limited it to 19,998 regular tokens, plus PAD and UNK. I used learned embeddings instead of pretrained ones. The table below shows the CNN and LSTM setups I used.

Component	CNN	RNN (bidirectional LSTM)
Embedding	64 dimensions; padding index 0	64 dimensions; padding index 0
Sequence encoder	Conv1d widths 3, 4, 5; 64 filters each; ReLU	One bidirectional LSTM; hidden size 64 per direction
Aggregation	Masked global maximum per filter; concatenate 192 features	Packed sequences; concatenate final forward/backward states (128 features)
Classifier	Dropout 0.3 and linear 3-class head	Dropout 0.3 and linear 3-class head

For the CNN, I excluded padding positions when taking the maximum across convolution outputs. For very short inputs, the convolution still uses a padded window. For the LSTM, I used packed sequences so padding would not count as part of the actual text. I trained both networks using cross-entropy loss on their output logits, with softmax only when probabilities were needed for prediction.

Training setting	Value
Optimizer	Adam; learning rate 0.001; weight decay 0.0001
Batch / gradient control	64 examples; gradient norm clipped at 1.0
Schedule / selection	Five epochs per condition; highest validation macro-F1 checkpoint
Randomness / compute	Seed 42; CPU; four PyTorch threads
Text budget	128 tokens maximum; combined: 63 prompt + separator + 64 response
Comparison control	Output-half condition: same first 64 response tokens as combined input

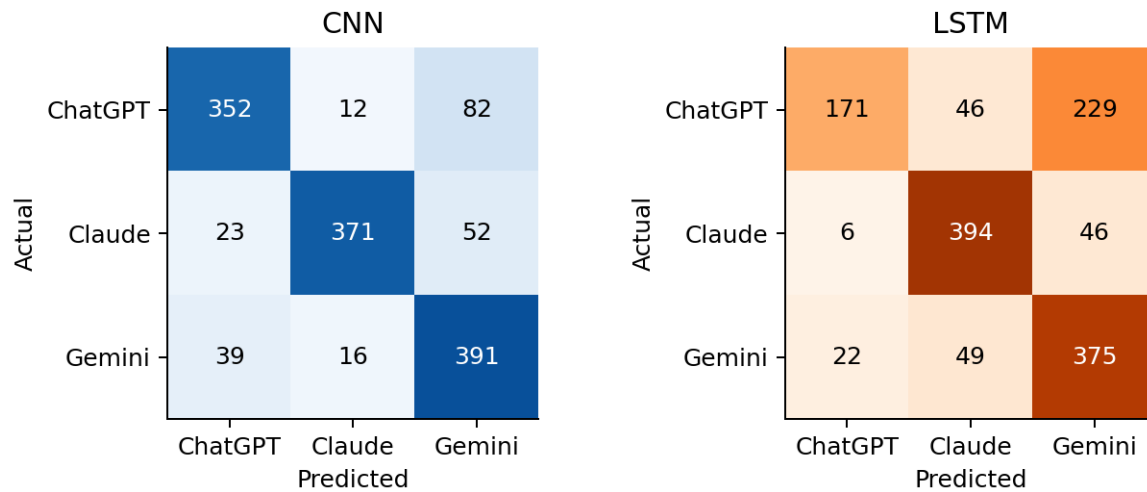


I kept the training settings fixed and selected checkpoints using validation macro-F1, not the test set. I used one random seed and five epochs for each condition because I wanted a consistent comparison across experiments. That also means I cannot say these are the best possible CNN or LSTM results. I saved the epoch histories and package versions so the runs could be checked later.

3. RQ1 - Can responses identify their source?

Model	Accuracy	Macro-F1	95% accuracy interval
CNN	83.3%	0.834	81.2% to 85.3%
LSTM	70.3%	0.686	68.1% to 72.5%
TF-IDF + logistic regression	92.7%	0.927	Not estimated

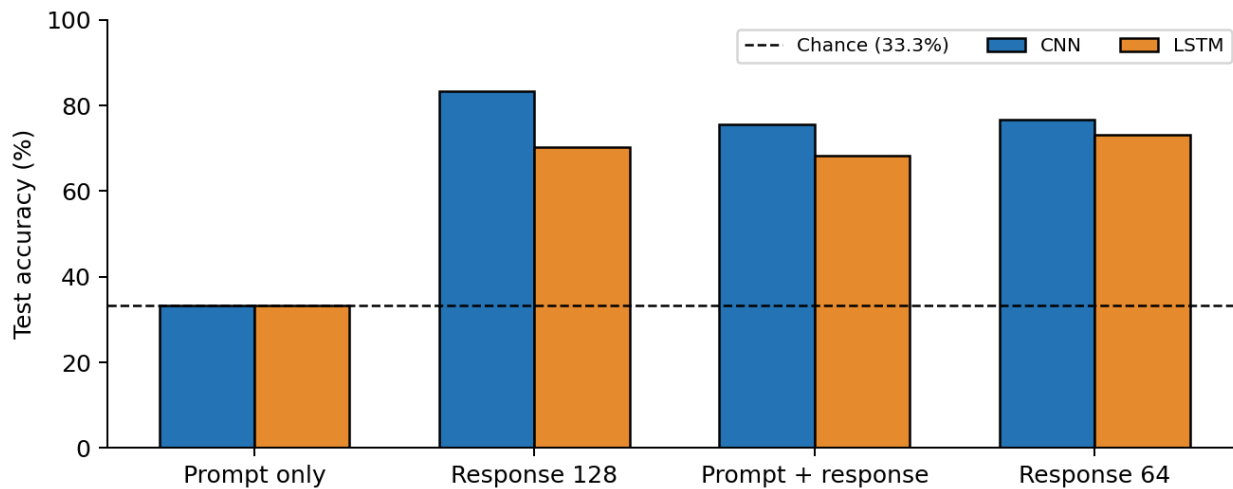
Because I had exactly 446 test examples from each LLM, a classifier that always guessed one label would get 33.3% accuracy. I reported macro-F1 as well as accuracy because it gives equal importance to all three families. To estimate uncertainty in accuracy, I resampled 1,000 groups of prompts, always keeping each prompt's three responses together. These intervals describe variation in this test set, not variation from retraining or using a different dataset.



Model / family	Precision	Recall	F1
CNN / ChatGPT	0.850	0.789	0.819
CNN / Claude	0.930	0.832	0.878
CNN / Gemini	0.745	0.877	0.805
LSTM / ChatGPT	0.859	0.383	0.530
LSTM / Claude	0.806	0.883	0.843
LSTM / Gemini	0.577	0.841	0.684

The first result that stood out to me was that both neural models performed well above chance using only response text. The CNN did better than the LSTM with the settings I used. This answers RQ1 for my test set: there is enough information in the responses to predict these dataset labels. It does not mean I verified which real-world model generated each answer. I also tried TF-IDF with logistic regression. It scored even higher, but it could use more of each response, so I would not treat that as a perfectly matched architecture comparison.

4. RQ2 - Does the user prompt help?



Condition	CNN accuracy / F1	LSTM accuracy / F1
Prompt only	33.3% / 0.323	33.3% / 0.290
Response only, 128 tokens	83.3% / 0.834	70.3% / 0.686
Prompt + response	75.7% / 0.757	68.4% / 0.685
Response only, 64 tokens	76.8% / 0.768	73.2% / 0.733

Both prompt-only models scored 33.3%. This makes sense once I look at how I built the data: each prompt appears with all three labels. If a model only sees that prompt, it cannot know which of the three responses it is supposed to identify. That is why the prompt-only result is at chance, even though the macro-F1 values are not exactly 0.333.

At first, I compared a combined prompt-and-response input with a response-only input of up to 128 tokens. But the combined version only has room for 64 response tokens, since it also includes 63 prompt tokens and a separator. That would not be a fair way to test the effect of the prompt. I therefore added another response-only experiment using the first 64 response tokens. In the table below, each difference means combined-input accuracy minus response-only accuracy.

Model / comparison	Accuracy difference	95% paired interval
CNN vs 128-response	-7.5 pp	-9.9 to -5.2 pp
CNN vs 64-response	-1.0 pp	-3.1 to +1.0 pp
LSTM vs 128-response	-1.9 pp	-4.9 to +1.1 pp
LSTM vs 64-response	-4.8 pp	-7.4 to -2.3 pp

I calculated the paired intervals using 2,000 bootstrap samples of prompt groups. For my CNN, adding the prompt changed accuracy by -1.0 percentage point compared with the matched 64-token response condition, and the interval included zero. For the LSTM, the change was -4.8 points and the interval stayed below zero. In these runs, I did not find evidence that adding the prompt helped. With a different, unbalanced dataset, prompt-only predictions might also reflect topics or data collection patterns rather than something about the LLM itself.

5. RQ4 - Which signals matter?

For RQ4, I wanted to understand what the models might be picking up beyond the overall accuracy score. I first compared some basic response statistics. Then I retrained each network in two modified conditions. In the plain-text condition, I removed punctuation and collapsed whitespace. In the name-masking condition, I replaced mentions of ChatGPT, OpenAI, Anthropic, Claude, Gemini, and Google with MODEL. I used the same splits and training budget each time.

Family	Mean words	Type/token ratio	Mean newlines	Bullet fraction
ChatGPT	243.8	0.624	0.0	0.0%
Claude	300.0	0.614	0.0	0.0%
Gemini	264.2	0.615	0.0	0.0%

I calculated the type/token ratio for each response and then averaged it within each class, so response length can influence that value. My bullet check looked for lists beginning at the start of a line. The dataset had almost no newlines and no detected bullets, which means those two measurements were not very helpful here. The tokenizer itself ignores whitespace changes, although it still uses punctuation as tokens.

Model / response condition	Accuracy	Macro-F1	Change vs original
CNN / output	83.3%	0.834	+0.0 pp
CNN / plain	74.4%	0.737	-8.9 pp
CNN / masked	82.6%	0.826	-0.7 pp
LSTM / output	70.3%	0.686	+0.0 pp
LSTM / plain	66.9%	0.668	-3.4 pp
LSTM / masked	73.8%	0.733	+3.6 pp

I found 54 responses, out of 8,901, that included at least one of the model or company names I screened. A name appearing in a response does not always mean that the model identified itself; it could simply be part of the topic. So I treated this as a rough check of name-related cues, not a perfect method for removing self-identification.

Paired change	CNN 95% interval	LSTM 95% interval
plain minus original	-11.1 to -6.6 pp	-6.4 to -0.4 pp
masked minus original	-2.2 to +0.7 pp	+1.3 to +5.9 pp

Removing punctuation reduced CNN accuracy by 8.9 percentage points and LSTM accuracy by 3.4 points. Masking model and company names lowered CNN accuracy by 0.7 points but increased LSTM accuracy by 3.6 points. I found the masking result interesting because it suggests those names were not the main reason the models worked. The plain-text results also suggest punctuation-related information may matter, especially for the CNN.

I have to be careful with that interpretation. Removing punctuation also shifts token positions and changes how much text fits into the sequence limit, so the lower accuracy does not prove that punctuation alone caused the difference. I also used only one training seed, which means some changes could be due to training variation. I see this experiment as a useful first look at possible style signals, rather than a complete explanation of the classifiers' decisions.

6. Conclusions, lessons and limitations

Answers supported by the experiments

RQ1: Yes, for the dataset I tested. My output-only CNN reached 83.3% accuracy and my LSTM reached 70.3%, both above the 33.3% baseline.

RQ2 (CNN): When I compared inputs with the same amount of response text, adding the prompt changed accuracy by -1.0 percentage point (95% paired interval: -3.1 to +1.0). I did not find a clear benefit from including the prompt.

RQ2 (LSTM): With the same matched-length comparison, adding the prompt changed accuracy by -4.8 percentage points (95% paired interval: -7.4 to -2.3). In this run, the response-only version worked better.

RQ4: My style summaries and the retrained plain-text and name-masking conditions gave me some clues about what might distinguish the dataset labels. I cannot use them to claim a universal LLM fingerprint. I left RQ3 for future work because the data lacked reliable domain labels.

Technical lessons from this workflow

The biggest lesson for me was that the way I prepare and split data can matter just as much as the model I pick. Keeping the three answers to each prompt together made the test more meaningful and helped me understand why prompt-only accuracy was exactly at chance. I also realized that comparing prompt-plus-response text with a full-length response was not fair, because the combined model had less room for the response itself. Adding the 64-token control made my answer to RQ2 more convincing. Finally, the logistic regression baseline reminded me that a more complicated neural network is not automatically the strongest classifier.

Working on this project also taught me a lot about actually running and checking experiments, rather than just reading about them. I had trouble getting the Python environment set up correctly on Windows, and it took me time to tell which files were old saved outputs and which ones were generated by my own run. Training all the conditions took patience, and I had to organize the results so I could use the correct numbers in the final report. I checked saved predictions, metrics, and other output files and learned why reproducibility matters. The most surprising part for me was that the CNN performed better than the LSTM, while the simple TF-IDF baseline performed better than both. I also learned to be careful about how I describe a result: a high accuracy on one dataset does not prove that I have discovered a fingerprint that works on every LLM. I used AI tools for help with debugging, organizing, and explaining parts of the workflow, but I ran and reviewed the experiments myself.

Limits and next experiments

There are several limits to what I can conclude from my results. I tested only one curated dataset and one random seed. I did not test newer model versions, unfamiliar LLM families, human-written text, or rewritten responses. I also do not know the exact generation settings for the original answers. Similar question templates, unusual text encoding, or dataset-specific patterns could make the classification problem easier than it would be in the real world.

If I continued this project, I would first run the models with more than one seed to see how stable the results are. I would also check a sample of the original texts and try a second independently collected dataset. For RQ3, I would need to assign and verify task categories before testing on entire domains that were missing from training. I did not carry out those additional experiments for this submission.

7. Reproducibility and references

My project folder contains the experiment and data-download scripts, prediction and report scripts, dependency list, team metadata, and the results_reproduction directory. The saved results include split identifiers, test predictions without raw text, class-level metrics, confusion matrices, training histories, and dataset counts. Checkpoints may exist locally from training, but the raw dataset is not included in this report. The reproduction steps below explain how to recreate the experiment outputs.

Run configuration

Item	Value
Python	3.12.7
PyTorch	2.14.1+cpu
NumPy / pandas	2.5.3 / 3.0.6
scikit-learn	1.9.1
Random seed / split seed	42 / 42
Epochs / sequence limit	5 / 128
Source archive SHA-256	ae8acb3e1d529bfa82938da9da4cfed72e34a5fff1bdf7a011a7612a716a3e0b

To repeat my workflow, install the packages listed in requirements.txt and run `python download_data.py`. Then run `python experiment.py -out results_reproduction --epochs 5 --max-len 128 --seeds 42`. Finally, run `python build_report.py --results results_reproduction`. I recorded the source archive hash above; if a later download has a different hash, it may not produce exactly the same results.

References and provided resources

[1] Course assignment screenshots supplied by the user: project overview, CNN/RNN requirement, research questions, evaluation rubric and project materials.

[2] Md. Sadiq Iqbal. TXD-22: A Large-Scale Benchmark Dataset. Kaggle, listed version 2. Dataset used.
<https://www.kaggle.com/datasets/mdsadiqiqbal/txd-22-a-large-scale-benchmark-dataset>

[3] PyTorch documentation. `torch.nn.Conv1d`. Implementation reference.
<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html>

[4] PyTorch documentation. `torch.nn.LSTM`. Implementation reference.
<https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

[5] PyTorch tutorial. NLP From Scratch: Classifying Names with a Character-Level RNN. Conceptual resource; our experiment uses word/punctuation tokens and an LSTM, not the tutorial name dataset.
https://docs.pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html

[6] LMSYS. Chatbot Arena Conversations. Instructor-suggested alternative considered but not used because access requires acceptance of its terms. https://huggingface.co/datasets/lmsys/chatbot_arena_conversations